

Algoritmos de planificación

Algoritmos básicos de planificación de la CPU

- Primero en llegar, primero en salir (FCFS – First come, first served)
- Primero el proceso más corto (SPN – Shorted process next)
- Menor tiempo restante (SRTF – Shortest remaining time first)
- Turno rotatorio o cíclico (RR – Round Robin)
- Planificación por prioridad (Derecho preferente)
- Planificación determinada por el comportamiento

Primero en llegar, primero en salir (FCFS – First come, first served)

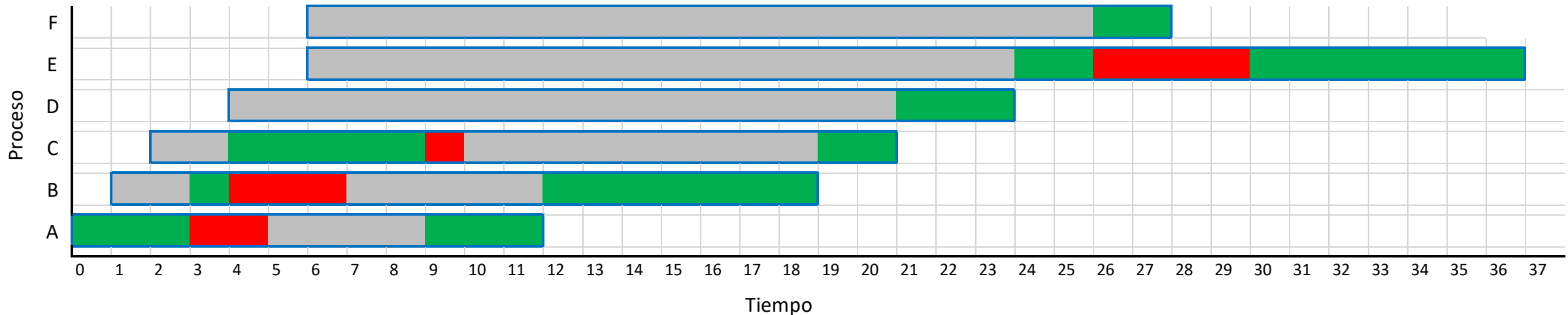
- Es una cola FIFO.
- La carga de trabajo se procesa simplemente en un orden de llegada.
- Por no tener en consideración el estado del sistema ni las necesidades de recursos de los procesos individuales, la planificación FCFS puede dar lugar a pobres rendimientos.
- Este algoritmo posee un alto tiempo de respuesta de la CPU pues el proceso no abandona la CPU hasta no haber concluido pues es un algoritmo Sin Desalojo (No Apropiativo).
- La planificación FCFS elimina la noción e importancia de las prioridades de los procesos.
- Para elegir el proceso al cual se le asignará la CPU, se escoge el que lleva más tiempo listo (primero en la cola).
- El proceso que se está ejecutando solo cede la CPU por dos razones:
 - Que se bloquee voluntariamente en espera de un evento. (Impresora, fichero, etc.)
 - Cuando termina su ejecución.
- La ventaja de este algoritmo es su fácil implementación, sin embargo, no es válido para entornos interactivos ya que un proceso de mucho cálculo de CPU hace aumentar el tiempo de espera de los demás procesos.

Primero en llegar, primero en salir (FCFS – First come, first served)

Para implementar el algoritmo sólo se necesita mantener una cola con los procesos listos ordenada por tiempo de llegada. Cuando un proceso pasa de bloqueado a listo se sitúa al final de la cola.

Existencia
Ejecución
Bloqueado
Espera

Proceso	Instante llegada li	Ejecución t	Bloqueo				Proceso	Ejecución t	Espera	Bloqueo	Instante fin lf	Retorno T = lf - li	Tiempo perdido T - t	Penalidad lp = T/t	Tiempo respuesta Tr
			inicio	duración											
A	0	6	3	2	Tiempo encendido	37,00	A	6	4	2	12	12	6	2,00	0
B	1	8	1	3	Uso total de CPU	35,00	B	8	7	3	19	18	10	2,25	2
C	2	7	5	1	CPU desocupada	2,00	C	7	11	1	21	19	12	2,71	2
D	4	3	0	0	Promedio de retorno	20,33	D	3	17	0	24	20	17	6,67	17
E	6	9	2	4	Promedio de ejecución	5,83	E	9	18	4	37	31	22	3,44	18
F	6	2	0	0	Promedio de espera	12,83	F	2	20	0	28	22	20	11,00	20
					Promedio de tiempo perdido	14,50									



Primero el trabajo más corto (SJF - Shortest Job First)

- Al igual que en el algoritmo FIFO las ráfagas se ejecutan sin interrupción, por tanto, sólo es útil para entornos batch (por lotes).
- Es un algoritmo no expulsivo (aunque se puede crear una variante preventiva).
- Cuando se activa el planificador, éste elige la ráfaga de menor duración. Es decir, introduce una noción de prioridad entre ráfagas.
- La ventaja que presenta este algoritmo sobre el algoritmo FCFS es que minimiza el tiempo de finalización promedio
- Hay que conocer de antemano la duración de cada trabajo
- Existe una posibilidad de inanición: Si continuamente llegan trabajos cortos, los trabajos largos nunca llegan a ejecutarse.
- Este algoritmo sólo es óptimo cuando se tienen simultáneamente todas las ráfagas

Primero el trabajo más corto (SJF - Shortest Job First)

El funcionamiento de este algoritmo viene dado por: Asociar a cada proceso el tiempo de ráfaga de CPU, luego se selecciona el proceso con menor ráfaga de CPU (en caso de empate aplicar FIFO).

Existencia

Ejecución

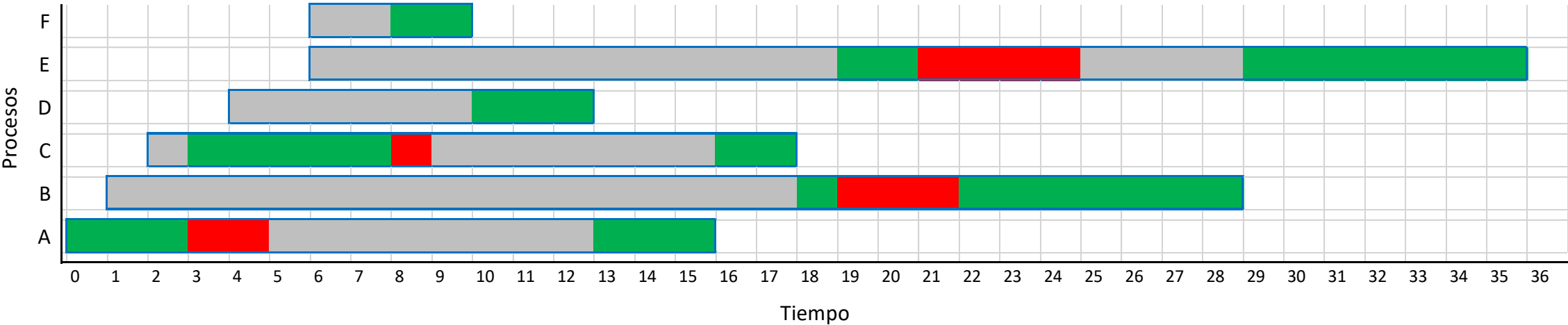
Bloqueado

Espera

Proceso	Instante llegada l_i	Ejecución t	Bloqueo	
			inicio	duración
A	0	6	3	2
B	1	8	1	3
C	2	7	5	1
D	4	3	0	0
E	6	9	2	4
F	6	2	0	0

Tiempo encendido	36,00
Uso total de CPU	35,00
CPU desocupada	1,00
Promedio de retorno	17,17
Promedio de ejecución	5,83
Promedio de espera	12,83
Promedio de tiempo perdido	11,33

Proceso	Ejecución t	Espera	Bloqueo	Instante fin l_f	Retorno $T = l_f - l_i$	Tiempo perdido $T - t$	Penalidad $l_p = T/t$	Tiempo respuesta T_r
A	6	8	2	16	16	10	2,67	0
B	8	18	3	29	28	20	3,50	17
C	7	10	1	18	16	9	2,29	1
D	3	10	0	13	9	6	3,00	6
E	9	23	4	36	30	21	3,33	13
F	2	8	0	10	4	2	2,00	2



Prioridad al tiempo restante más corto (SRTF, Short Remaining Time First)

Es similar al anterior, con la diferencia de que si un nuevo proceso pasa a listo se activa el dispatcher para ver si es más corto que lo que queda por ejecutar del proceso en ejecución. Si es así, el proceso en ejecución pasa a listo y su tiempo de estimación se decrementa con el tiempo que ha estado ejecutándose

Existencia

Ejecución

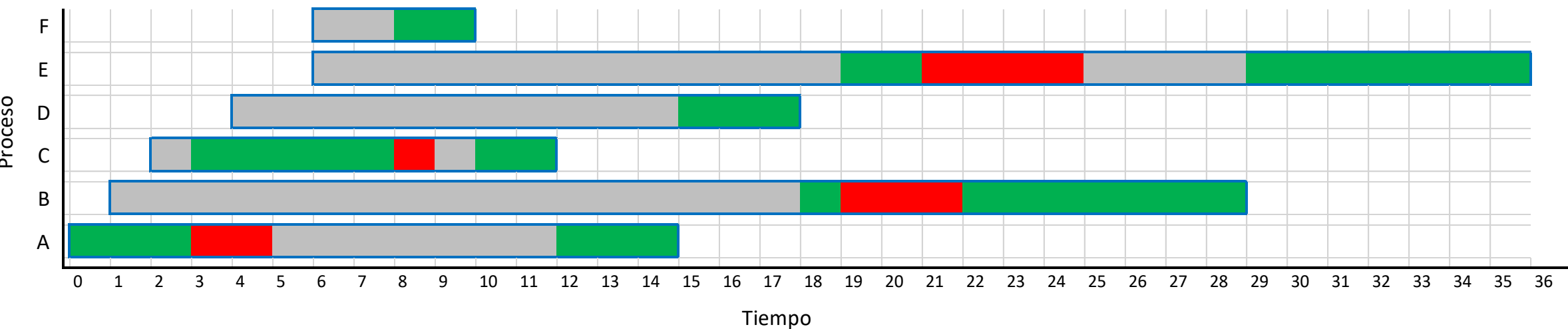
Bloqueado

Espera

Proceso	Instante llegada l_i	Ejecución t	Bloqueo	
			inicio	duración
A	0	6	3	2
B	1	8	1	3
C	2	7	5	1
D	4	3	0	0
E	6	9	2	4
F	6	2	0	0

Tiempo encendido	36,00
Uso total de CPU	35,00
CPU desocupada	1,00
Promedio de retorno	16,83
Promedio de ejecución	5,83
Promedio de espera	12,50
Promedio de tiempo perdido	11,00

Proceso	Ejecución t	Espera	Bloqueo	Instante fin l_f	Retorno $T = l_f - l_i$	Tiempo perdido $T - t$	Penalidad $l_p = T/t$	Tiempo respuesta T_r
A	6	7	2	15	15	9	2,50	0
B	8	18	3	29	28	20	3,50	17
C	7	4	1	12	10	3	1,43	1
D	3	15	0	18	14	11	4,67	11
E	9	23	4	36	30	21	3,33	13
F	2	8	0	10	4	2	2,00	2



Turno rotatorio o cíclico (RR – Round Robin)

- Este es uno de los algoritmos más antiguos, sencillos y equitativos en el reparto de la CPU entre los procesos, muy válido para entornos de tiempo compartido. Algoritmo apropiativo.
- Mantiene una cola (FIFO) con los procesos listos para ser ejecutados o continuar su ejecución
- Un proceso recibe al procesador durante una fracción de tiempo (quantum). Si hay n procesos en la cola de listos y el quantum es q , cada proceso recibe $1/n$ del tiempo de CPU en intervalos de q unidades de tiempo como mucho. Ningún proceso espera más de $(n-1)q$ unidades de tiempo. Un proceso regresa a la cola de listos cuando:
 - Expira su ranura de tiempo
 - Concluye el evento que lo llevó a la cola de bloqueados
- Un proceso pasa a la cola de bloqueados cuando espera a que se efectúe un evento.
- A veces se asigna una prioridad a cada proceso, bajándose ésta cada vez que se le da turno (planificación con retroalimentación).
- Al ocurrir una interrupción de reloj que coincide con la agotación del quantum se llama al **dispatcher**, generando retraso (quantum de tiempo para cambiar de contexto).
- Cambio de contexto lleva tiempo. Compromiso entre eficiencia del procesador y velocidad de respuesta:
 - q grande $\rightarrow$ FCFS
 - q pequeño $\rightarrow$ q debe ser grande con respecto a la conmutación de contexto, en otro caso la sobrecarga es muy alta

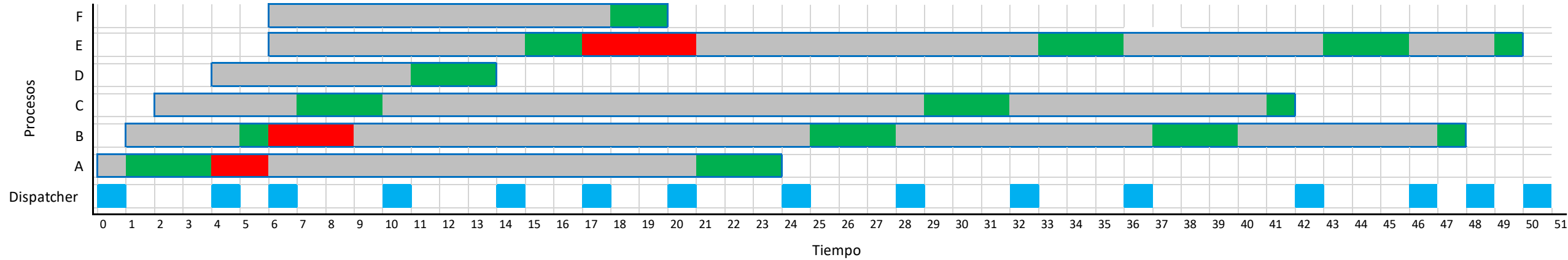
Turno rotatorio o cíclico (RR – Round Robin)

Proceso	Instante llegada l_i	Ejecución t	Bloqueo	
			inicio	duración
A	0	6	3	2
B	1	8	1	3
C	2	7	5	1
D	4	3	0	0
E	6	9	2	4
F	6	2	0	0
Q = 3				

Tiempo encendido	51,00
Tiempo procesos	35,00
Tiempo sistema	16,00
% CPU procesos	68,63
% CPU sistema	31,37
Promedio de retorno	29,83
Promedio de ejecución	5,83
Promedio de espera	27,17
Promedio de tiempo perdido	24,00

Proceso	Ejecución t	Espera	Bloqueo	Instante fin l_f	Retorno $T = l_f - l_i$	Tiempo perdido $T - t$	Penalidad $l_p = T/t$
A	6	18	2	24	24	18	4,00
B	8	40	3	48	47	39	5,88
C	7	35	1	42	40	33	5,71
D	3	11	0	14	10	7	3,33
E	9	41	4	50	44	35	4,89
F	2	18	0	20	14	12	7,00

Despachador
Existencia
Ejecución
Bloqueado
Espera

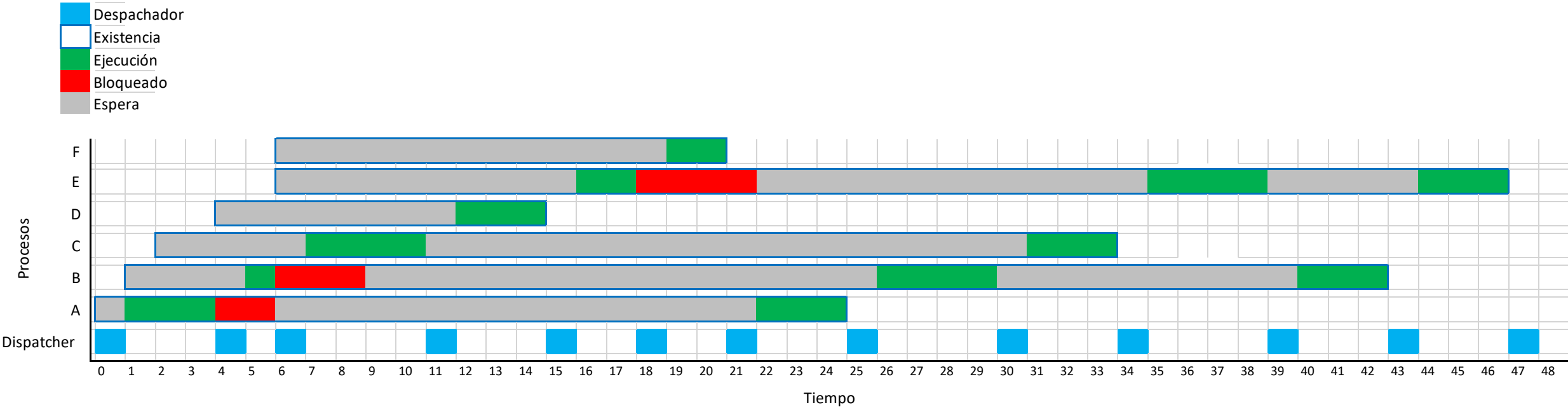


Turno rotatorio o cíclico (RR – Round Robin)

Proceso	Instante llegada l_i	Ejecución t	Bloqueo	
			inicio	duración
A	0	6	3	2
B	1	8	1	3
C	2	7	5	1
D	4	3	0	0
E	6	9	2	4
F	6	2	0	0
Q = 4				

Tiempo encendido	48,00
Tiempo procesos	35,00
Tiempo sistema	13,00
% CPU procesos	72,92
% CPU sistema	27,08
Promedio de retorno	27,67
Promedio de ejecución	5,83
Promedio de espera	25,00
Promedio de tiempo perdido	21,83

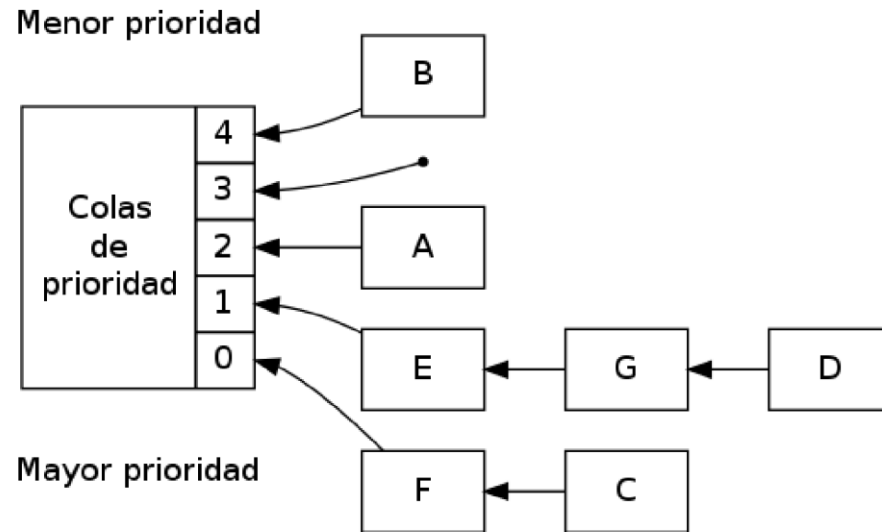
Proceso	Ejecución t	Espera	Bloqueo	Instante fin l_f	Retorno $T = l_f - l_i$	Tiempo perdido $T - t$	Penalidad $l_p = T/t$
A	6	19	2	25	25	19	4,17
B	8	35	3	43	42	34	5,25
C	7	27	1	34	32	25	4,57
D	3	12	0	15	11	8	3,67
E	9	38	4	47	41	32	4,56
F	2	19	0	21	15	13	7,50



Planificación por prioridad (Derecho preferente)

- Se asignan prioridades a los procesos
- Se agrupan los procesos con la misma prioridad en colas
- Se selecciona primero el proceso de más alta prioridad
- Alternativas:
 - Prioridades fijas, pero causa problemas de inanición
 - Solución: prioridades dinámicas por mecanismos de envejecimiento (bajar la prioridad a un proceso cada vez que se le da el turno).
- Se pueden agrupar por prioridades y utilizar turno rotatorio para cada prioridad
- Asignación estática o dinámica
 - Una opción puede ser ejecutar el proceso de mayor prioridad completamente y no se interrumpe hasta que se finalice o bloquee, o llegue otro de mayor prioridad... etc.
 - Muy usado en sistemas de tiempo real

Planificación por prioridad (Derecho preferente)



Tenemos dos posibilidades:

- Expropiativo
- No expropiativo

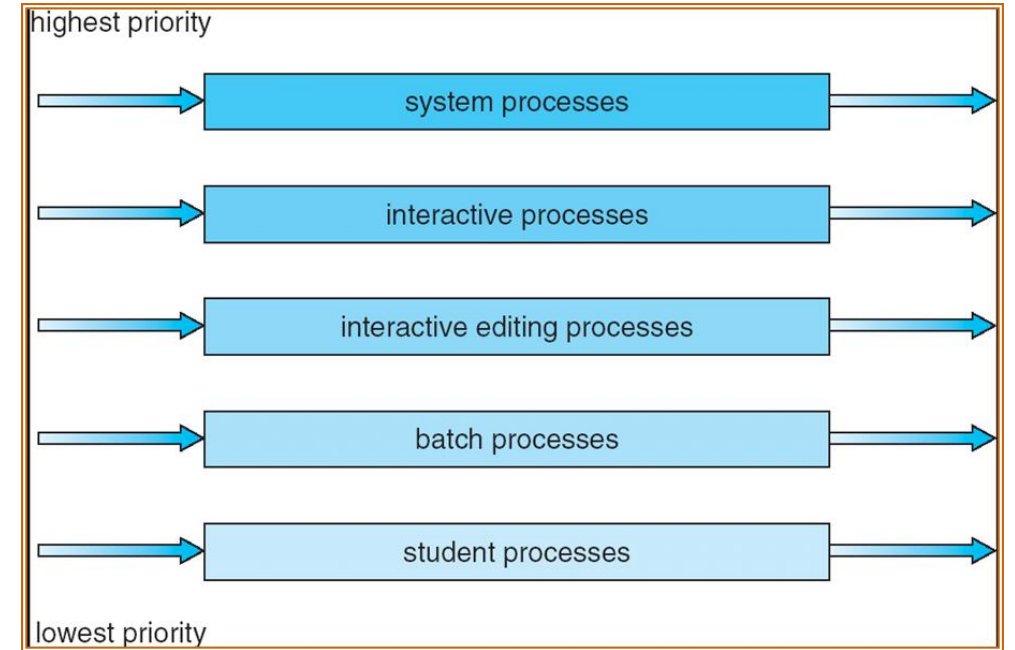
- En los anteriores algoritmos la planificación se hace sobre una única cola. Para este tipo de planificación lo mejor es agrupar los procesos con prioridades similares en colas.
- La prioridad es un número entero: mientras menor sea este entero pues mayor prioridad tiene el proceso.
- Para determinar la prioridad de un proceso, existe una disciplina de planificación no apropiativa en la cual la prioridad de cada proceso no solo se calcula en función del tiempo de servicio (tiempo en la CPU) sino también del tiempo que ha esperado para ser atendido.

$$\text{Prioridad} = \frac{\text{tiempo de espera} + \text{tiempo de servicio}}{\text{tiempo de servicio}}$$

- Con esta fórmula notamos que como el tiempo de servicio está en el denominador los procesos cortos tendrán preferencia, y como también el tiempo de espera aparece en el numerador, los procesos largos que también han esperado, tendrán una prioridad favorable. Sin embargo, cada sistema operativo determinará su propia disciplina para calcular y gestionar las prioridades.

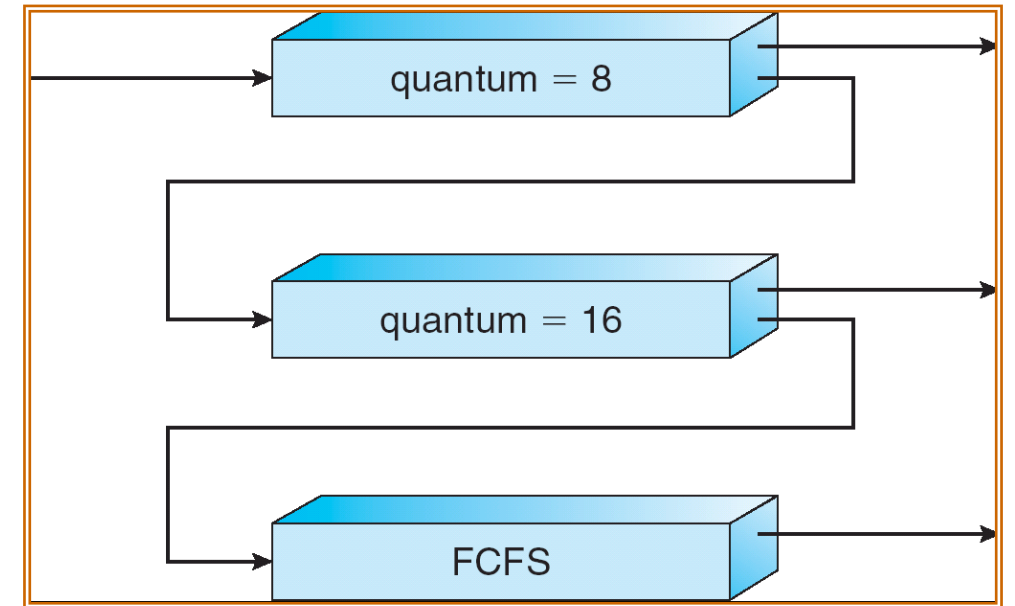
Algoritmo de Colas Multinivel

- La cola de listos se divide en colas separadas.
Ej.:
 - procesos de primer plano (interactivos)
 - procesos de segundo plano (por lotes)
- Cada cola puede tener un algoritmo de planificación diferente
 - primer plano – RR
 - segundo plano – FCFS
- Se debe planificar a nivel de cola
 - Planificación por prioridad fija; ej.: la cola de primer plano tiene prioridad sobre la de segundo plano. Posible inanición.
 - División de tiempo – cada cola obtiene cierta porción de tiempo de CPU que reparte entre sus procesos; ej., 80% para la cola de primer plano (RR) y 20% para la de segundo (FCFS)



Algoritmo de Colas Multinivel con retroalimentación

- En este caso un proceso se puede mover entre las colas. Es una forma de implementar el envejecimiento para evitar inanición.
- Un algoritmo de planificación de colas multinivel con realimentación está definido por los siguientes parámetros:
 - número de colas
 - algoritmos de planificación para cada cola
 - método usado para determinar cuándo promover un proceso a una cola de mayor prioridad
 - método usado para determinar cuándo degradar un proceso a una cola de menor prioridad
 - método usado para determinar en qué cola ingresará un proceso cuando necesite servicio



Otros mecanismos de planificación

- Los algoritmos presentados pueden caracterizarse sobre dos descriptores primarios:
 - Tipo de multitarea, si el esquema está planteado para operar bajo multitarea preventiva o cooperativa
 - Emplea información intrínseca, si para tomar cada decisión de planificación emplean información propia (intrínseca) a los procesos evaluados, o no.
- Los esquemas de planificación empleados normalmente usan mezclas de los algoritmos presentados (esquemas híbridos), ya que al emplear un algoritmo puede que presente más ventajas ante una situación dada y evitar algunas de sus deficiencias
- En general, los diseñadores de un sistema operativo implementarán un algoritmo de planificación que crea conveniente para el uso al que está orientado.

Planificación en Windows

Por lo general en Windows se utiliza una política de planificación: **Round Robin**. La desventaja principal es que cambia los procesos en ejecución con demasiada frecuencia (Lo que supone una pequeña pérdida de tiempo).

Además, las prioridades en Windows se organizan en dos bandas o clases: tiempo real y variable. Cada una de estas bandas consta de 16 niveles de **PRIORIDAD**.

- El algoritmo es de Colas Multinivel con Realimentación. Cada prioridad tiene asociada una cola con planificación RR.
- Prioridades 0-15 variables, 16-31 fijas (tiempo real).
- A los hilos que agotan su quantum se les reduce la prioridad. Cuando un hilo pasa de espera a listo se aumenta su prioridad

Modificadores (hilos)		real-time	high	above normal	normal	below normal	idle priority
	time-critical	31	15	15	15	15	15
	highest	26	15	12	10	8	6
	above normal	25	14	11	9	7	5
	normal	24	13	10	8	6	4
	below normal	23	12	9	7	5	3
	lowest	22	11	8	6	4	2
	idle	16	1	1	1	1	1

Planificación en Linux

Linux está basado en la planificación tradicional de Unix añadiendo 2 clases de prioridad para procesos de tiempo real y tiempo compartido flexibles. Las clases de planificación de Linux son las siguientes:

SCHED_OTHER : Es la planificación clásica de UNIX. Examina las prioridades dinámicas (calculadas como combinación de las especificadas por el usuario y las calculadas por el sistema). Los procesos que llevan más tiempo en el sistema van perdiendo prioridad.

- Prioridad basada en créditos – el proceso con más créditos es el siguiente en tomar la CPU
- Los créditos se reducen cuando ocurre una interrupción de reloj
- Cuando el crédito es 0, se escoge otro proceso
- Cuando todos los procesos tienen crédito 0 se asigna de nuevo crédito para todos los procesos (Basado en factores como prioridad e historia)

SCHED_FIFO : El sistema FIFO o FCFS (First to Come is First Served). Los procesos esperan en cola y se ejecutan secuencialmente. Se sigue calculando un quantum de tiempo para el proceso, generalmente no se usa porque con esta planificación no se fuerza al proceso a abandonar la CPU. Se usa en procesos de tiempo real.

- Tiempo real blando (Cumple el estándar Posix.1b) – dos clases: FCFS y RR
- El proceso de mayor prioridad siempre se ejecuta primero

Sin embargo, a partir del Kernel 5.6 cambiaron el algoritmo y al nuevo le llaman **algoritmo más justo**.

Cada Sistema de Planificación tiene sus ventajas y desventajas, al fin y al cabo cada uno se usa según la necesidad de cada sistema.